

OOP Task 2 – Complete Solution

Question 1: Search

1.1 Abstract Classes vs Interfaces

What is an abstract class, and what makes a method abstract in Python?

An abstract class is a class used as a blueprint for other classes. In Python, it can be created using ABC and abstractmethod from the abc module. A method is abstract when it is decorated with @abstractmethod. A concrete subclass must implement its abstract methods before it can be instantiated.

What is partial abstraction vs full abstraction?

Partial abstraction means a class can contain both abstract methods and implemented methods. Python does not have a dedicated interface keyword like Java; ABCs and typing.Protocol can be used to express interface-like designs.

Does Python have a dedicated interface keyword?

No. Python has no dedicated interface keyword. Developers commonly use Abstract Base Classes (ABCs) with @abstractmethod, or typing.Protocol for structural/interface-like behavior.

What happens if you instantiate a class that has unimplemented abstract methods?

Python raises a TypeError. The class cannot be instantiated until all required abstract methods are implemented in a concrete subclass.

Example:

```
from abc import ABC, abstractmethod
```

```
class Animal(ABC):
    @abstractmethod
    def make_sound(self):
        pass

    def sleep(self):
        print("Animal is sleeping")
```

```
class Dog(Animal):
    def make_sound(self):
        print("Woof")
```

1.2 Duck Typing and Polymorphism in Python

What is duck typing, and where does the name come from?

Duck typing focuses on what an object can do rather than its declared type. The name comes from the idea: if an object walks like a duck and quacks like a duck, it can be treated like a duck.

How does duck typing let different objects be used interchangeably without common inheritance?

A function can call the required method directly. Any object that provides that method can be passed to the function, even if the objects do not share a parent class.

How is duck typing different from method overriding?

Method overriding usually happens when a subclass inherits from a parent class and replaces an inherited method with its own implementation. Duck typing does not require inheritance; it only requires compatible behavior.

Give an example of duck typing instead of strict type-checking.

A function can call obj.speak() without checking whether obj is a Dog, Cat, or another specific class.

```
class Dog:
    def speak(self):
        print("Woof")

class Robot:
    def speak(self):
        print("Beep")

def make_sound(obj):
```

```
obj.speak()
```

```
make_sound(Dog())  
make_sound(Robot())
```

Question 2: Garage Management System

The Garage class implements `add_car()`, `remove_car()`, `park_car()`, and `display_available_spots()`. Encapsulation is used by making capacity, available spots, and the car list private with double underscores.

```
class Garage:
    def __init__(self, capacity):
        self.__capacity = capacity
        self.__available_spots = capacity
        self.__cars = []

    def add_car(self, car):
        if self.__available_spots > 0:
            self.__cars.append(car)
            self.__available_spots -= 1
            print(f"{car} added to the garage.")
        else:
            print("Garage is full.")

    def remove_car(self, car):
        if car in self.__cars:
            self.__cars.remove(car)
            self.__available_spots += 1
            print(f"{car} removed from the garage.")
        else:
            print("Car is not in the garage.")

    def park_car(self, car):
        self.add_car(car)

    def display_available_spots(self):
        print(f"Available spots: {self.__available_spots}")
```

Question 3: ElectricCar

ElectricCar inherits from both Car and Battery, demonstrating multiple inheritance. Because the task does not specify a battery-to-range conversion, the implementation assumes 1% charge equals 5 km.

```
class Car:
    def __init__(self, brand):
        self.brand = brand

    def drive(self):
        print("Driving the car")

    def show_info(self):
        print(f"Car brand: {self.brand}")

class Battery:
    def charge(self):
        print("Battery is charging")

    def check_range(self, current_charge):
        # Assumption: 1% battery charge = 5 km.
        range_km = current_charge * 5
        print(f"Estimated remaining range: {range_km} km")

class ElectricCar(Car, Battery):
    def __init__(self, brand):
        Car.__init__(self, brand)

# Example:
car = ElectricCar("Tesla")
car.show_info()
car.drive()
car.charge()
car.check_range(80)
```